Particle Swarm Optimization (PSO) Algorithm for Network Resource Allocation

Step 1: Define the Problem

- Objective function: Maximize total network utility $U = \sum_{i=1}^{N}$

Bafna Gold
Date: Page:

Mathematical Formulation

1. Optimization Space Dynamics

$x_i = [x_{i,1}^t, x_{i,2}^t, \ldots, x_{i,M}^t]$ and

$V_i^t = V_{i,1}^t, V_{i,2}^t, \ldots, V_{i,M}^t]$

$M \rightarrow$ No of network users
$x_i^t \in \Omega \rightarrow$ position vector
$V_i^t \in R^M \rightarrow$ velocity vector

2. Kinematic State Update Equations

$$V_{i,j}^{t+1} = w \cdot V_{i,j}^t + c_1 \cdot r_1 \cdot (pbest_{i,j} - x_{i,j}^t) + c_2 \cdot r_2 \cdot (gbest_j - x_{i,j}^t)$$

$$x_{i,j}^{t+1} = x_{i,j}^t + V_{i,j}^{t+1}$$

3. Objective Function & Constrained Fitness

Maximize $U(x_i^t) = \sum_{j=1}^{M} \ln(x_{i,j}^t)$

Subject to $\sum_{j=1}^{M} x_{i,j}^t \leq R_{max}$

To handle the linear inequality constraint, fitness function incorporates an exterior penalty method:

$$F(x_i^t) = U(x_i^t) - \lambda \cdot \max\left(0, \sum_{j=1}^{M} x_{i,j}^t - R_{max}\right)^2$$

$\lambda >> 1$ is the penalty parameter

PSO Algorithm for Resource Allocation

step 1. Set Bounds & Space
Define search domain $\Omega \subset R^M$ for M users
Set boundaries $x_{i,j} \in [X_{min}, X_{max}]$ and total capacity hyperplane constraint
$\sum_{j=1}^{M} x_{i,j} \leq R_{max}$ (total available network capacity

step 2 Initialize Parameters
set swarm size N, max iterations T, inertia w and acceleration constants C1, C2

step 3 Instantiate states (t = 0)
For each particle $i \in \{1 \dots N\}$ randomly sample starting coordinates:

$$x_{i,j}^0 \sim U(X_{min}, X_{max}), \quad v_{i,j}^0 \sim U(-V_{max}, V_{max})$$

$$pbest_i = x_i^0$$

step 4. Compute fitness & Memory

Evaluate each particle position via the penalized objective function ($\lambda \gg 1$)

$$f(x_i^t) = \sum_{j=1}^{M} \ln(x_{i,j}^t) - \lambda \cdot \max\left(0, \sum_{j=1}^{M} x_{i,j}^t - R_{max}\right)^2$$

(M proportional terms) (penalty)

local update: If $F(x_i^t) > F(pbest_i)$ =)

$$pbest_i = x_i^t$$

Global Update: $gbest = \text{argmax}_{pbest_i} F(pbest_i)$

Step5: Kinematic Update & Boundary Projection.

Sample random weights $r_1, r_2 \sim U(0,1)$.
Compute next state mechanics across all dimensions j.

→ pulls towards pbest

$$v_{i,j}^{t+1} = w \cdot v_{i,j}^{t} + c_1 \cdot r_1 \cdot (pbest_{i,j} - x_{i,j}^{t}) + c_2 \cdot r_2 \cdot (gbest_j - x_{i,j}^{t})$$

↳ gbest

$$x_{i,j}^{t+1} = \Pi_{[x_{min}, x_{max}]}(x_{i,j}^{t} + v_{i,j}^{t+1}) = \max(x_{min}, \min(x_{max}, x_{i,j}^{t} + v_{i,j}^{t+1}))$$

↳ Euclidean projection operator

Step6: Iteration Advancement Loop.

Increment $t = t+1$. If $t >= T$ or $\|gbest^{t} - gbest^{t-1}\| < \varepsilon$, proceed to step 7, or else repeat from step 4. threshold

Step7. Output optima
Return gbest as the optimal resource vector & F(gbest) as the max network utility.

Proceed to step 7

o/p convergence achieved early at iteration 01

Optimal Allocation vector (gbest):
[21.22 33.29 17.28 28.11] Mbps
Total Combined Resource o/p: 99.91/100. Mbps
Maximum Network Utility F(gbest): 12.7462

```python
import random
import math

# Step 1: Set Bounds and Space
M = 4
xmin = 1
xmax = 50
Rmax = 100

# Step 2: Initialize Parameters
N = 30
T = 100
w = 0.7
c1 = 1.5
c2 = 1.5
Vmax = 10
lam = 100

# Step 3: Initialize Particles
particles = []
velocities = []
```

```
for i in range(N):
    x = [random.uniform(xmin, xmax) for j in range(M)]
    v = [random.uniform(-Vmax, Vmax) for j in range(M)]

    particles.append(x)
    velocities.append(v)

pbest = [x.copy() for x in particles]

# Fitness Function
def fitness(x):
    utility = sum(math.log(value) for value in x)
    penalty = lam * max(0, sum(x) - Rmax) ** 2
    return utility - penalty

# Initialize Global Best
gbest = max(pbest, key=fitness).copy()

# Step 4, 5 and 6: PSO Iteration Loop
for t in range(T):

    for i in range(N):

        # Update Velocity
        for j in range(M):
            r1 = random.random()
```

```
        r1 = random.random()
        r2 = random.random()

        velocities[i][j] = (
            w * velocities[i][j]
            + c1 * r1 * (pbest[i][j] - particles[i][j])
            + c2 * r2 * (gbest[j] - particles[i][j])
        )

        # Limit velocity
        velocities[i][j] = max(
            -Vmax, min(Vmax, velocities[i][j])
        )

    # Update Position
    for j in range(M):
        particles[i][j] += velocities[i][j]

        # Boundary Projection
        particles[i][j] = max(
            xmin, min(xmax, particles[i][j])
        )

    # Update Personal Best
    if fitness(particles[i]) > fitness(pbest[i]):
        pbest[i] = particles[i].copy()
```

```
            if fitness(particles[i]) > fitness(pbest[i]):
                pbest[i] = particles[i].copy()

        # Update Global Best
        new_gbest = max(pbest, key=fitness).copy()

        if abs(fitness(new_gbest) - fitness(gbest)) < 1e-8:
            gbest = new_gbest
            break

        gbest = new_gbest

    # Step 7: Output Results
    print("Optimal Allocation Vector:", [round(x, 2) for x in gbest])
    print("Total Allocated Resource:", round(sum(gbest), 2), "Mbps")
    print("Maximum Network Utility:", round(fitness(gbest), 4))
```

```
Optimal Allocation Vector: [28.89, 16.36, 21.96, 31.94]
Total Allocated Resource: 99.16 Mbps
Maximum Network Utility: 12.7117
```